Salesforce Services

A custom cartridge developed for Red Van Workshop and Autobahn for accessing other Salesforce clouds, such as Marketing and Sales/Service, from Commerce Cloud

October 2021
v 1.0
Damian Dobosz

Table of Contents

Overview

This documentation is for a custom cartridge created to provide easy to consume, generic code for accessing other Salesforce Clouds from Salesforce Commerce Cloud (also referred to as SFCC), as well as guidance as to how and where to incorporate additional Salesforce API calls and how and when to implement custom code for clients that will utilize this cartridge.

The initial release of this cartridge handles both Salesforce Marketing (also referred to as SFMC) and Salesforce Sales/Service (also referred to as SFDC) Clouds. In the future, other Salesforce Clouds or specific contexts of use, such as for the Salesforce Order Management System (aka SOM), may be added.

This documentation will cover the configuration of the destination clouds; SFMC and SFDC, in at least enough detail to be able to connect and communicate with those clouds. However, it is beyond the scope of this document to go into great detail for the configuration and setup of features in the other clouds such as creating journeys within SFMC or custom objects within SFDC.

Version one of this cartridge contains support for some of the most commonly used APIs in both SFMC and SFDC. It is not meant to have comprehensive coverage of all of the available APIs. It is designed with the idea that as various API calls are found to be needed, those calls can be added to the cartridge.

Because the reader may only be concerned with interacting with one of the clouds, the documentation is broken up into cloud specific sections, as opposed to single sections covering all the Site Preferences, all the Services, etc.

Additionally, most of the examples shown in the SFMC and SFDC API sections are in the context of working with a single cloud. There are some notes in those sections, as well as in the SFMC and SFDC Client and Cartridge extension sections, that include guidance for when both clouds are used by a client.

Metadata

The Salesforce Services cartridge has metadata that will need to be installed into the instance that will be using it. The contents of the metadata includes Site Preferences, Services, Custom Object Definitions and Instances.

Cartridge Design & Architecture

There are two primary classes (files) that are main entry points for the cartridge and, for most use cases, should be all that is needed (to be “required”) to interact with to communicate with SFMC and SFDC. These “manager” classes have been constructed in an attempt to achieve the following goals:

- Encapsulate from the consumer, as much as possible, the infrastructure needed for authenticating, setting up the service and request, making the call and processing the response
- Create a reusable cartridge that can satisfy the needs of any client needing to communicate with SFMC and SFDC as well as providing the framework for easily adding new API calls, extending and combining API calls along with how and where to make client specific customizations.
- Provide an easier to use, less complicated and less fragile cartridge as an alternative to the Salesforce provided connector cartridges for SFMC and SFDC (that also are likely deprecated)
- Allow consumers of this cartridge to be able to focus their resources on understanding and formatting the data that will be sent to Marketing and Service Clouds instead of on the nuts and bolts of how the communication happens

Manager Classes

The above referenced entry point classes are named “MarketingManager” and “ServiceManager”.

The purpose of these classes is to expose generic operations against either SFMC or SFDC; for example, validating an email address, executing a triggered send or firing an event in a journey in Marketing Cloud. In Service Cloud it could be querying for a Lead (object) by email address or creating an object in SFDC.

The specific functionality mentioned above will be covered in greater detail in the API sections for each cloud, however, the goal here is to emphasize that the functionality exposed in these managers is and should remain centered on actions one would perform against SFMC or SFDC and not to encompass SFCC and/or client

specific needs . There will likely be the need for specific client customizations; for example, inserting a customer's address into a specific data extension in Marketing Cloud after that customer does XXX.
See the API section for each cloud for the guidance on when to add to and when to extend the manager classes.

Each function within the manager classes follow the same pattern:

- Retrieve the Service
- Setup the request and potentially the service (header and url modifications)
    - This process includes authentication
- Call the API
    - Check the response for an invalid access token and re-authenticate and call the API again, if necessary
- Process the Response

The caller of each function is expected to pass in data ready to be passed to SFMC or SFDC and will receive back a JSON object. The response data may just be the Salesforce response parsed into their object structure and/or it may have some additional processing; e.g. a normal HTTP 200 response that contains error messages.

Support Classes

There are a handful of other files in the cartridge that support the functionality provided through the manager classes. There are only a couple of these classes that should ever need to be extended, and they are summarized below.

Authentication Helper

This class is responsible for exactly what its name implies. Changes or extensions should not be necessary for this file until Salesforce modifies their authentication process for one of the clouds.

Each cloud has its own similar, but different, process for constructing the HTTP header and body for authentication, as well as the response that is returned is not quite the same. This class handles all those differences. The consumer of this cartridge should never have to interact with this class.

Marketing & Service Request Factories

These classes are also doing what their name implies as well as potentially modifying the HttpService object. The services objects sometimes need modifications as some of the API endpoints include a specific ID in the URL or have to have the RequestHeader altered; i.e. “GET”, “DELETE”, “PATCH” (“POST” is default).

The functions in the request factories also follow a pattern, which is:

- Retrieve the Access Token and Add to the Header
    - If the token is missing/expired, a new one will be retrieved via the AuthenticationHelper
- Update the URL for the version and potentially other additions
- Any additional HTTP header modifications or additions
- Build and return the object for the body of the request (if needed)

These classes will need additions as new API calls are supported.

Marketing & Service Response Factories

Like the AuthenticationHelper, these classes are doing only what their name suggests. The functions here follow a pattern as well:

- Check if the response is good HTTP response (valid)
- (Potentially) check if that good response contains error message
- Parse the Salesforce response into an object
- Returned a structured response indicating success or failure, containing the data returned from the call or a meaningful error message

There is a “Generic” response handler for each cloud. As new API calls are added to the cartridge, depending on what is in the response, additional functions may or may not be needed in these classes; the generic response handler may suffice.

Marketing Cloud

SFMC Configuration

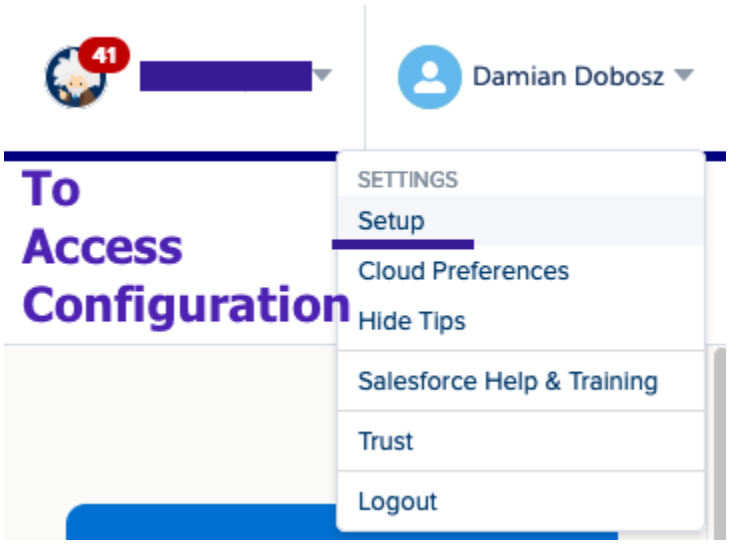
In order for Commerce Cloud to communicate with Marketing Cloud, the API integration has to be configured and enabled on SFMC.

Doing this requires administrative rights. Depending on the client, their internal resources, and the state of their SFMC instance, this may already be completed and an admin at the client simply provides the necessary information.

However, as is often the case with smaller Autobahn clients, the SFMC instance may not have the API set up yet. If this is the case, then the following steps will be a guide for configuring SFMC. If it is already configured, then you can jump to the end of the In SFMC section for the information on what you need from the client admin.

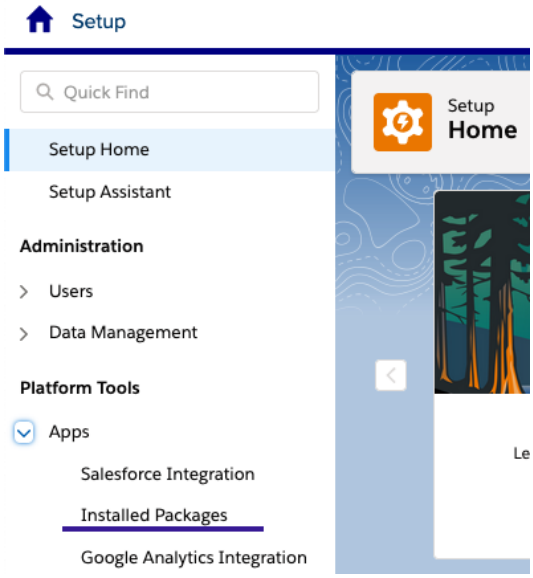
In SFMC

In order to be able to access existing configuration information or perform the configuration in SFMC, your account will need access to the Setup option. Administrative rights are needed in order to see this option.



Need access to the Setup option for configuration in SFMC

From the Setup menu, select the Installed Packages option

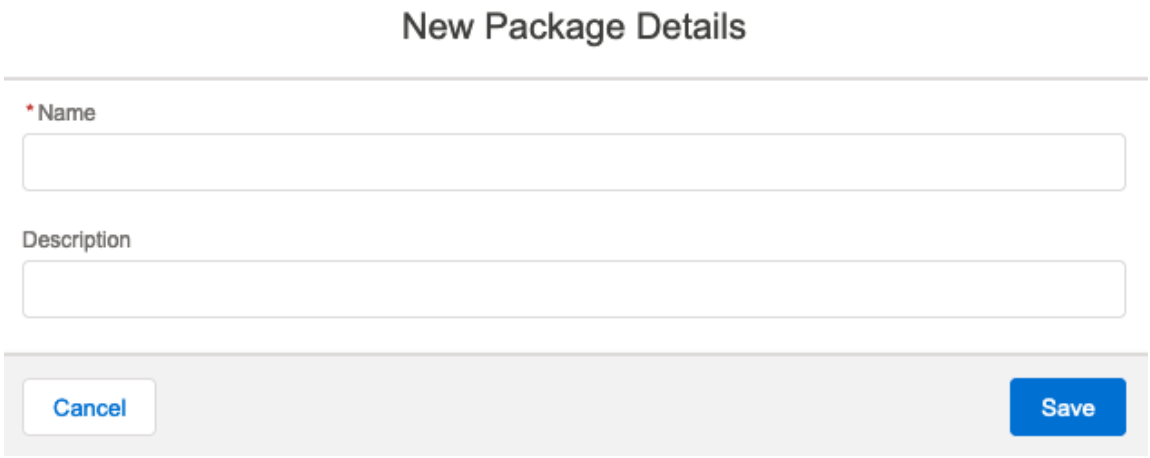


SFMC Setup Menu - Platform Tools

Once added and configured, you would see the package listed.

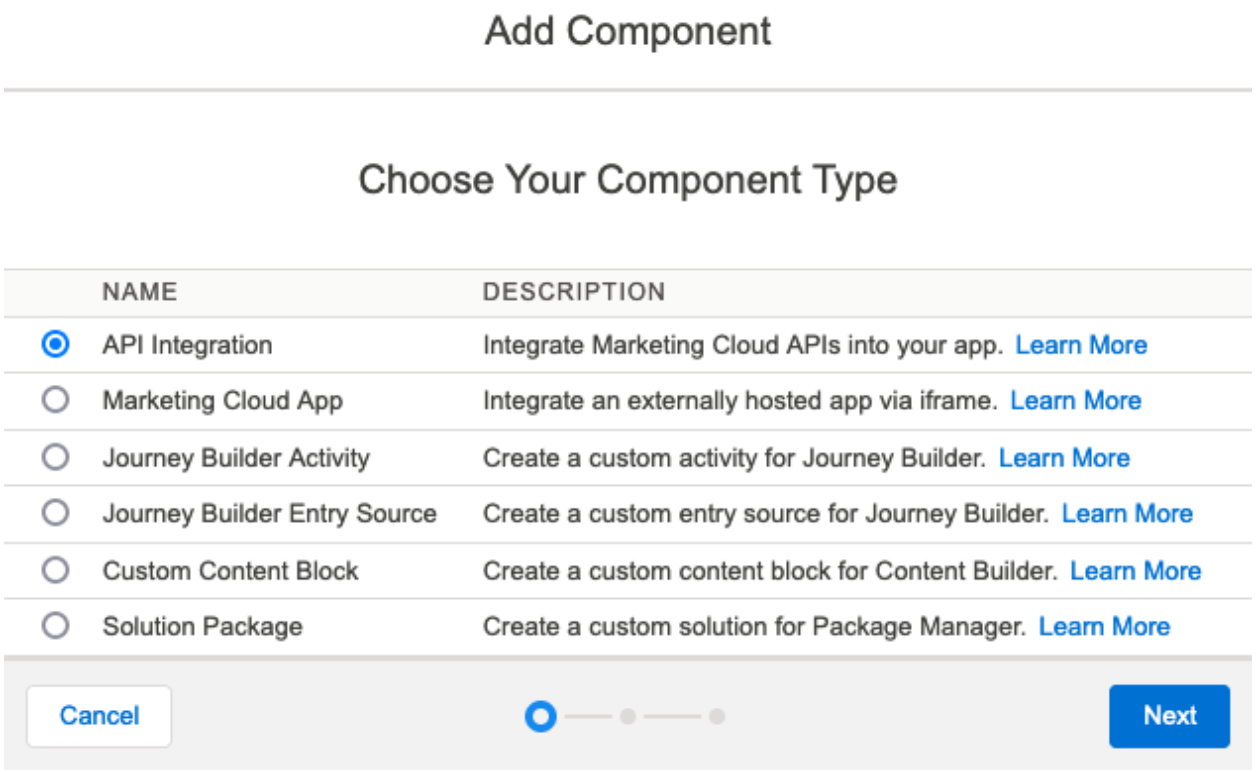
Example of an API integration already configured and listed in SFMC

If, however, it has not already been configured, a package will need to be created. Select to add a new package and give it an appropriate name and description.



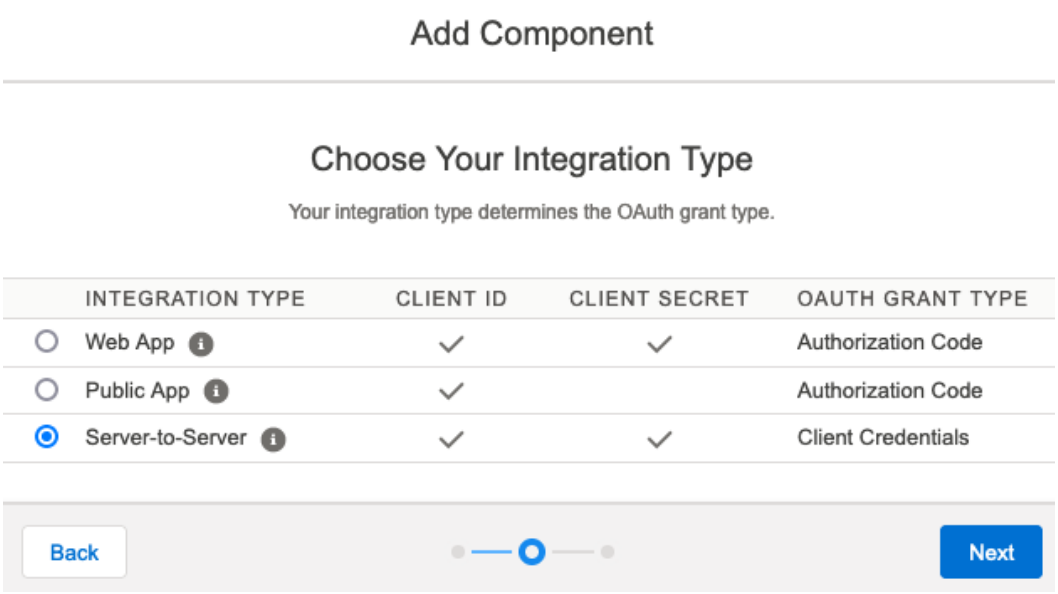
The new package details in SFMC

Once the package is created, the API Component will need to be added to it.



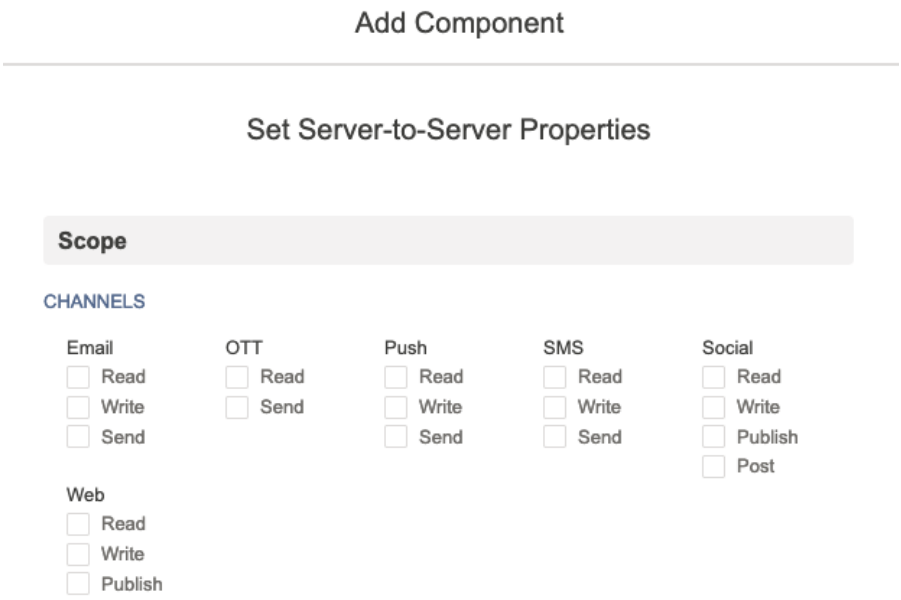
Adding the API Integration component to an SFMC package

Choose the “Server-to-Server” integration type.



API Component Integration Type selection in SFMC

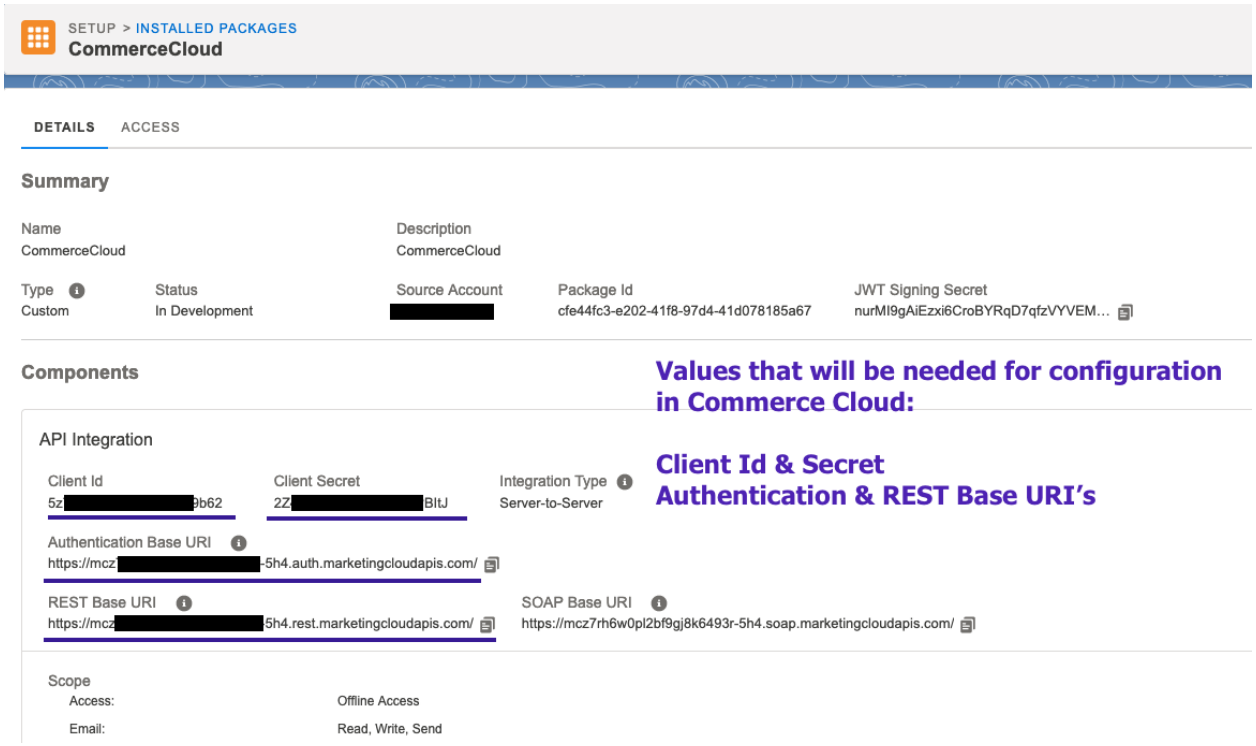
Then select the scope (permission set) to add to the component. These can be edited later, so there’s no need to worry about not setting the right scope and then not being able to add or remove later.



Setting the scope for the API Integration in SFMC

The suggested starting point for scope is for subscribers read/write, email read/write/send, anything for data extensions, and all the journey options.

Once complete, the details of the API integration / package will have the necessary data points that will be needed for Commerce Cloud.



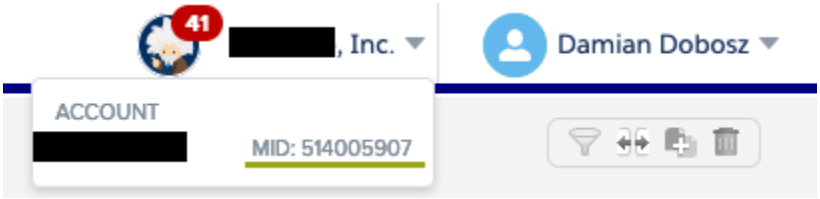
API Integration Package Details in SFMC

SFMC API Integration Configuration Data Points

The data points needed by SFCC are:

- Client Id
- Client Secret
- Authentication Base URI
- REST Base URI
- Version
- Account Id

The account id, also known as the MID, can be found in the upper right hand corner right next to the current user's name and menu options.



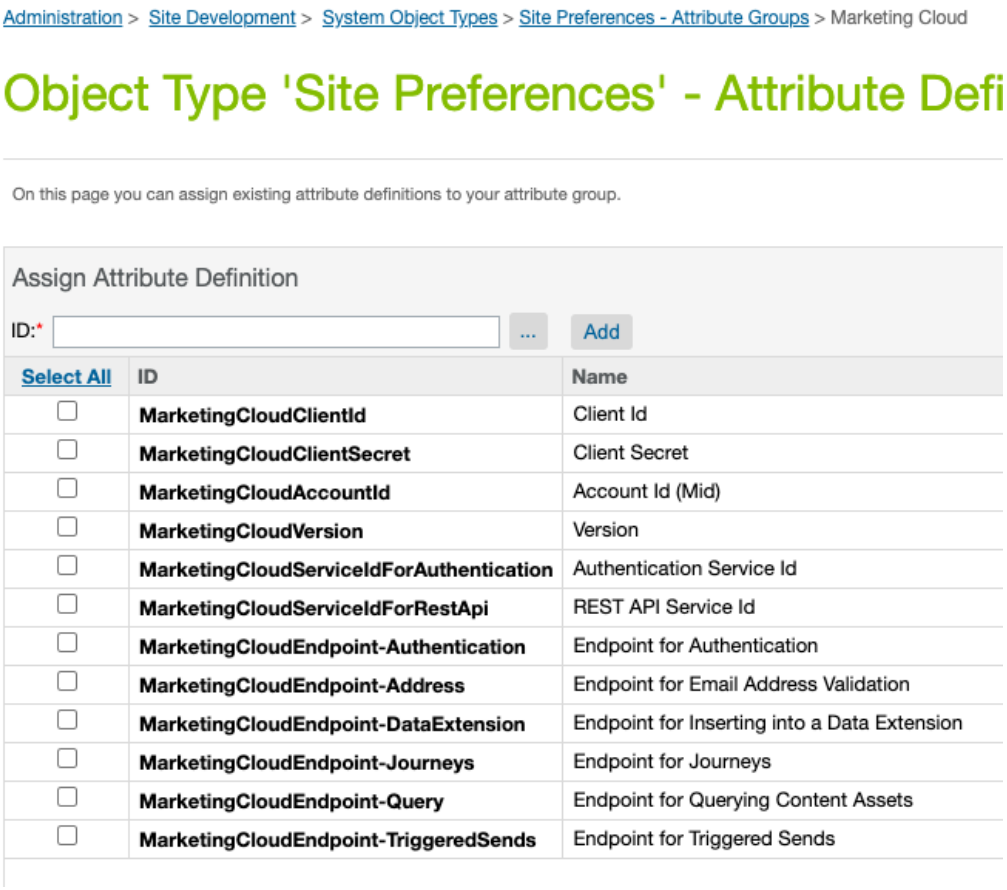
Account Id (MID) of an SFMC instance

SFMC In SFCC

In order to use the cartridge to access SFMC from SFCC, a few site preferences need values from SFMC.

SFMC Site Preferences

Of the 12 site preferences for Marketing Cloud, likely only 3 or 4 will need values for the specific SFMC instance that will be accessed.



SFMC related Site Preferences

Those are:

- MarketingCloudClientId
- MarketingCloudClientSecret
- MarketingCloudAccountId
- (Potentially) MarketingCloudVersion

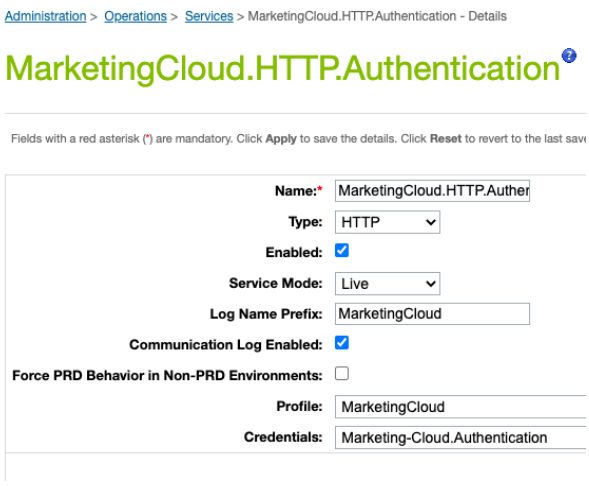
The value for the version will only need a value added over the default at some point in the future when Salesforce updates the SFMC API and the version number that should be specified changes; e.g.. from “1” to “2”.

The remaining site preferences have default values that likely will not need to be changed unless there is an update to how the relative endpoint is structured in a future update to the SFMC API. For example, if the authentication endpoint were to change from “/v2/token” to “/v3/oauth3”.

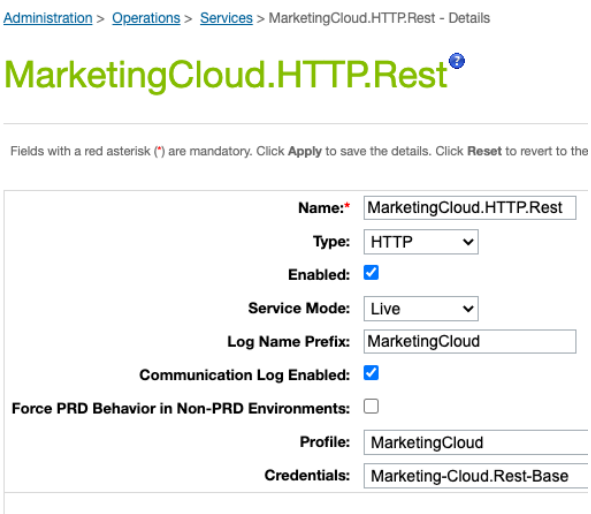
The site preferences for the ServiceIds should also not need to be changed as any configuration changes should be handled at the Service. See the next section for more details.

SFMC Services

Access to SFMC is handled by two service definitions in SFCC:



SFMC Authentication Service

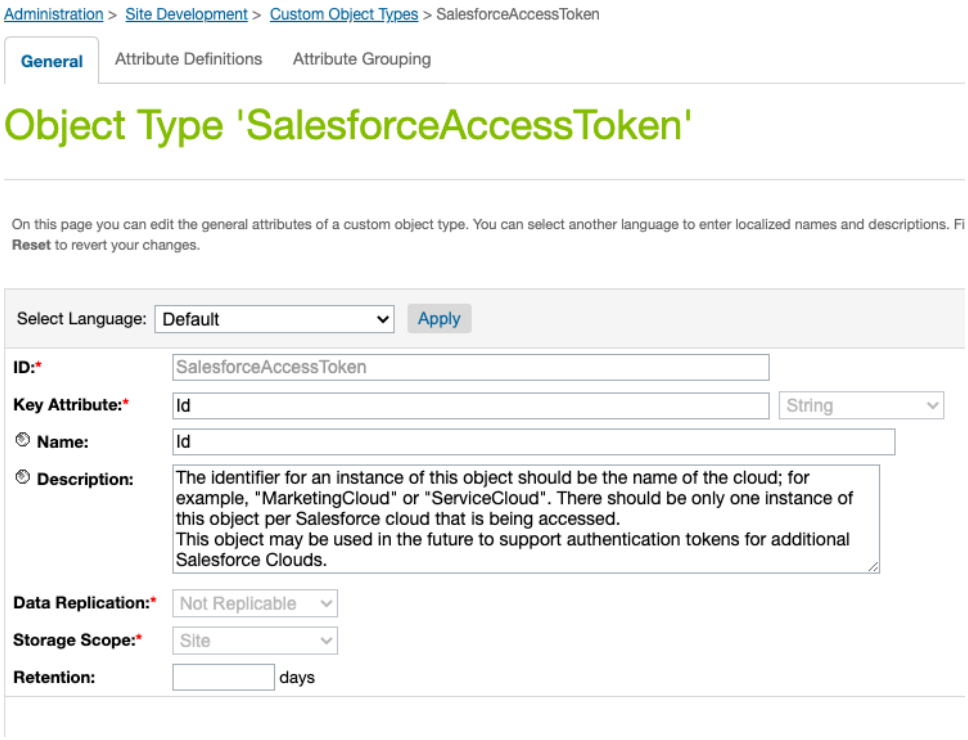


SFMC REST Service

If an SFCC instance should need to be switched to communicate with different SFMC instances, then a second credential object could be created with another URL. For example, included with the cartridge is a credential called “Marketing-Cloud.Rest.Base”. A second credential called “Marketing-Cloud.Rest.Base.Alternate” could be created with the different URL. This seems unlikely at this time as there are not “Development”, “Staging” and “Production” instances for SFMC as there are for SFCC.

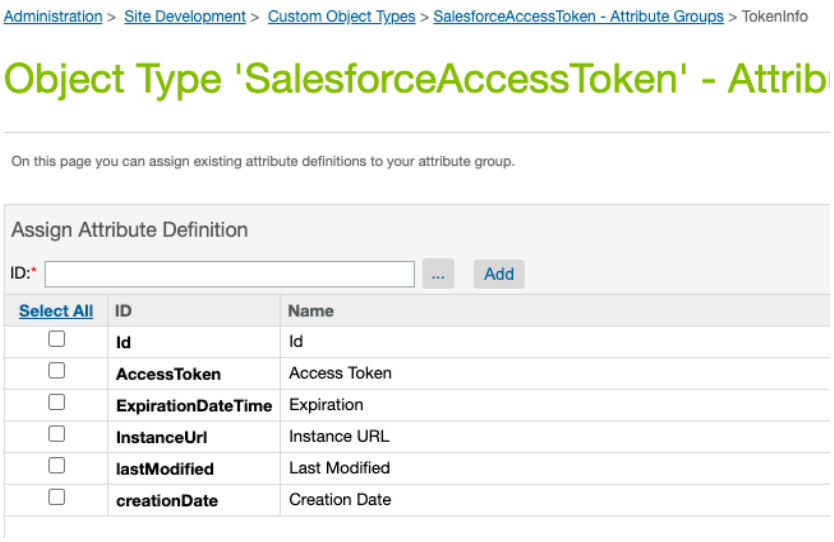
SalesforceAccessToken Custom Object

The metadata for the cartridge will also add the definition of a new custom object called “SalesforceAccessToken”.



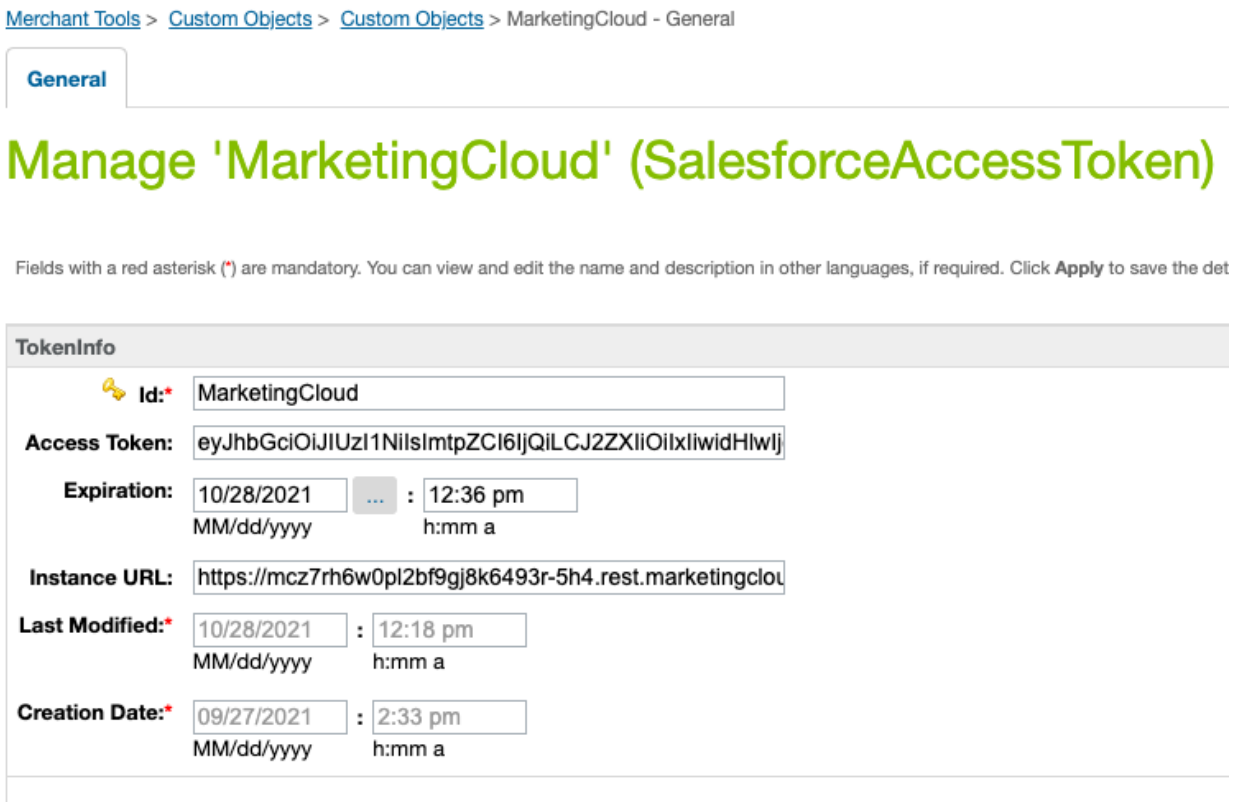
Definition of the SalesforceAccessToken custom object

The attributes of the custom object are:



SalesforceAccessToken custom object attributes

An instance of this object on a site once the cartridge is in use will look like this.



An instance of the SalesforceAccessToken object for SFMC

When the metadata for this cartridge is imported, it will create an empty instance for the SalesforceAccessToken object for Marketing Cloud. It must exist prior to using the cartridge, as storefront requests in SFCC cannot create instances of custom objects, and an instance of it is needed to store the access token SFMC returns during the authentication process.

SFMC Supported API

The following sections detail which API calls are currently supported and may include additional information on common use cases, examples of calling them and possible extension points.

For the examples in all of the following sections, this is the require statement that would also be needed:

```
var marketingManager = require('*/cartridge/scripts/marketing/MarketingManager');
```

SFMC Authentication

There should be no need to directly interact with SFMC authentication. It may be worth knowing that the access token returned from SFMC will expire. The cartridge will automatically handle retrieving a new token when it is needed.

Address Validation

The address validation API call is a simple one needing only to be passed an email address and will be returned a relatively simple response indicating that it is valid or is not, and potentially what reason it is not.

Common Use Case: Client wants to validate an email address before adding it to their newsletter list.

Example Calling

```
let addressResponse =
marketingManager.ValidateEmailAddress('damian@redvanworkshop.com');
if (addressResponse.Successful && addressResponse.SalesforceInfo.valid) {
      // ... do more stuff
```

Possible Client Extension

A foreseeable client side extension might be combining this call with another to SFMC. For example, in a “MarketingManager” file in the client’s core cartridge, first call this function to validate the email address, then if valid, call the function to insert it into a Data Extension.

Inserting into a Data Extension

This API call does exactly what you think it does; adds the given data to the Data Extension. The caller of this API is expected to pass data to this function that aligns with the construction of the data extension.

Common Use Case
The customer has indicated that they want to be notified when a product comes back in stock/launches/etc.

Example calling

```
let item1 = {
      SubscriberKey: "damian@redvanworkshop.com",
      EmailAddress: "damian@redvanworkshop.com",
      FirstName: "Damian",
      LastName: "Dobosz"
};

let items = [];
items.push(item1);

let insertResponse =
marketingManager.InsertIntoDataExtension('NewProductXXXExtension', items);
```

Note: The above example is in the context of a Marketing Cloud only scenario. If a client had both SFMC and SFDC, the above shown “SubscriberKey”, instead of being an email address, might be a “Contact ID” or “Lead ID”.
When constructing the data object that will be sent to SFMC, it is always a good idea to reach out to the client’s SFMC Admin or the RVW SFMC Architect assigned to the client to confirm how data is being mapped.

Possible Client Extension
A possible extension here might be to wrap the above example into client’s “MarketingManager” file in their core cartridge to simplify the call for developers on the project.

```
function addNewCustomerToEmailList(customer) {
      let dataExtensionName = getDataExtensionFromSitePrefereces();
      let customerInfo = getDataForExtension(customer);
      let insertResponse =
      marketingManager.InsertIntoDataExtension(dataExtensionName,
      customerInfo);
}
```

Journeys

The only supported API call for journeys, at the moment, is to fire an event defined within a journey.

Common Use Case

The classic example here would be the “Welcome” journey that a customer would begin once they have created an account on the site.

Example calling

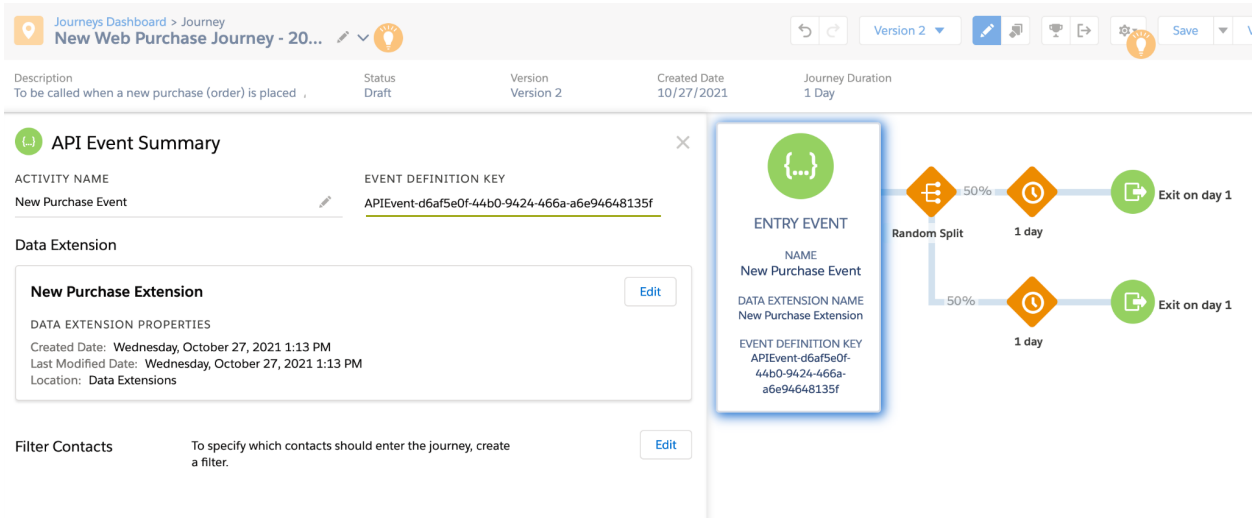
```
function startCustomerWelcomeJourney(customer) {
      let customerInfo = {
            SubscriberKey: customer.email,
            EmailAddress: customer.email,
            FirstName: customer.firstName,
            LastName: customer.lastName
      };
      // TODO: Get this key from a Site Preference
      let eventDefinitionKey = 'APIEvent-d6af5e0f-...-466a-a6e94648135f';
      let journeyResponse =
      marketingManager.FireJourneyEvent(eventDefinitionKey, customerInfo);
}
```

Possible Client Extension

As the above example suggests, adding a customized function to the client’s core cartridge “MarketingManager” that encapsulates getting the event definition key and building the data object.

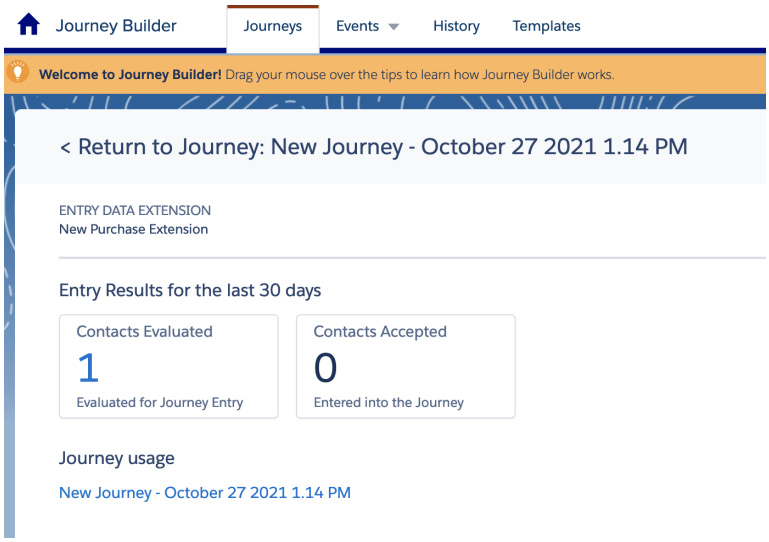
Journey Event within SFMC

Without getting into the details of how journeys are built in SFMC, the following screenshots may give some insight into the process.



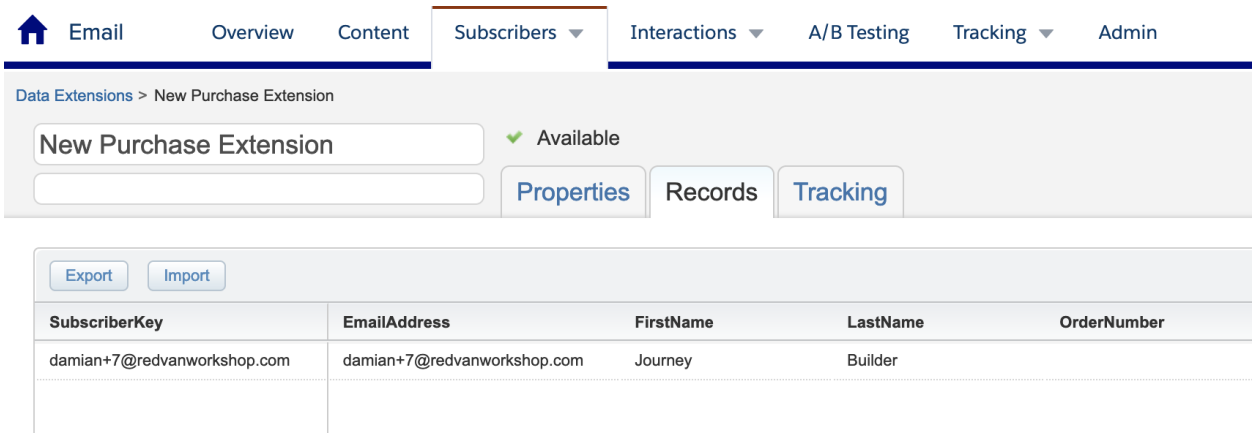
A Journey and it's API Event Summary in SFMC

After firing the event, what happens / is visible on the SFMC side:



The entry into the Journey

This screenshot would also apply to the “Insert into Data Extension” API call.



The addition to the Data Extension

Executing Triggered Sends

A Triggered Send within SFMC is typically setup for a specific action by a customer. This could be completing a purchase, requesting to reset their password, etc. This API call will pass the given data to SFMC along with which Triggered Send this applies to, which is identified by the external key.

Common Use Case

Any transactional email such as Order Confirmation, Password Reset, etc.

Example calling

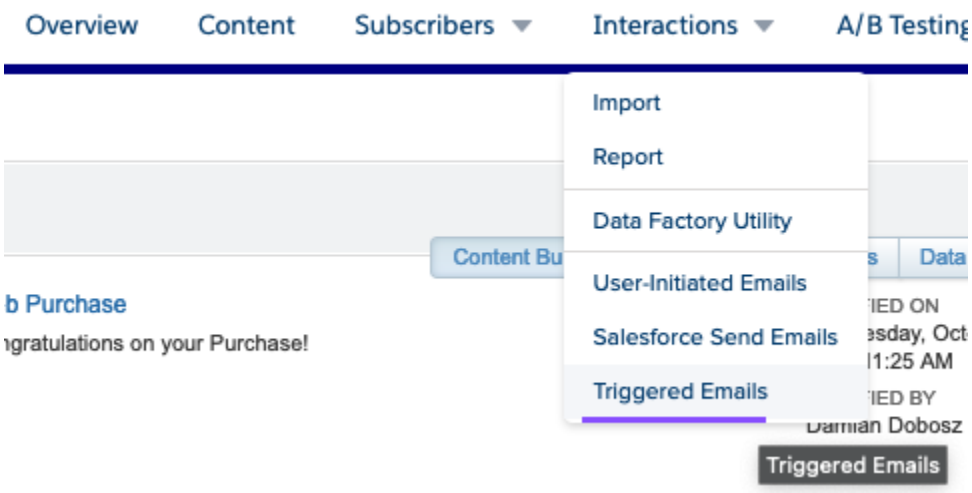
```
let customerInfo = {
      SubscriberKey: "damian@redvanworkshop.com",
      EmailAddress: "damian@redvanworkshop.com",
      FirstName: "Damian",
      LastName: "Dobosz"
};

let sendResponse = marketingManager.ExecuteTriggeredSend('OrderConfirmation',
customerInfo);
```

Possible Client Extension

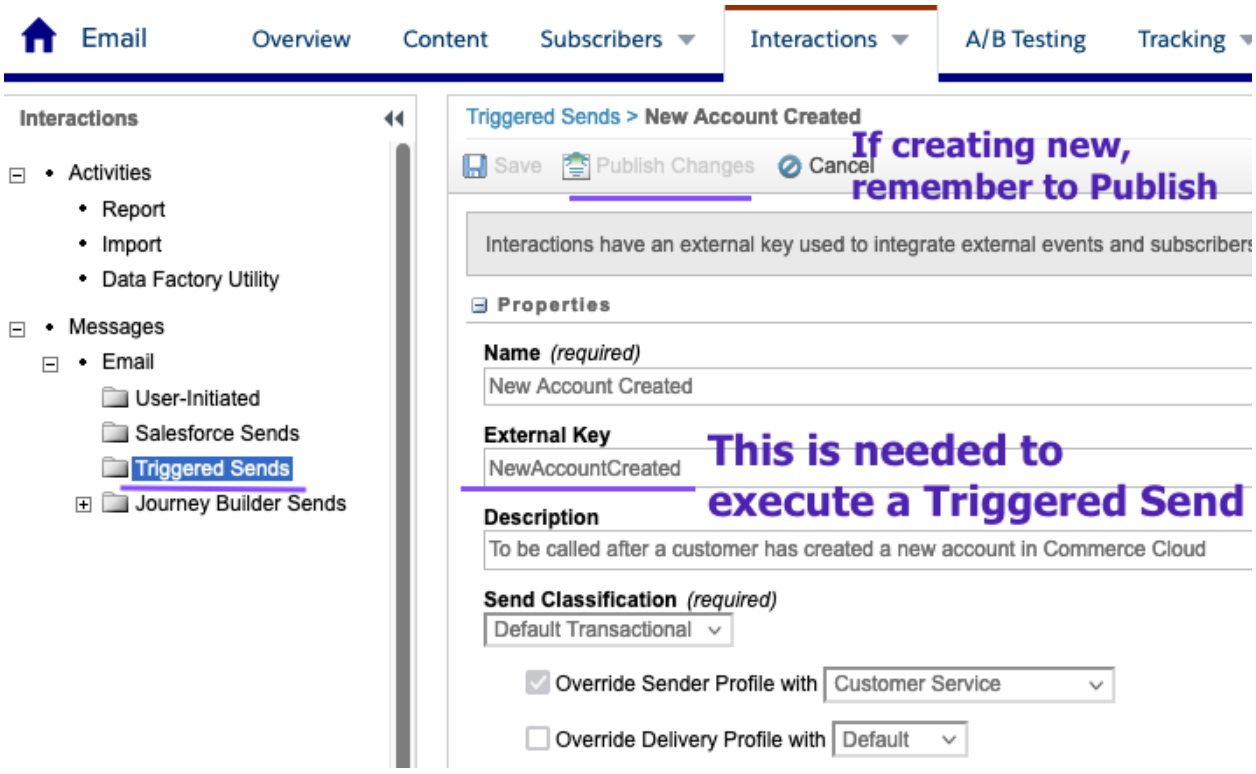
A customized function to the client's core cartridge "MarketingManager" that encapsulates getting the external key and building the data object. For example, in the Order Confirmation example this might mean getting product information from the order to add to the data passed to SFMC.

Triggered Send within SFMC

Without getting into the details of how Triggered Sends are constructed in SFMC, the following screenshots may give some insight into the process.



The Interactions menu, Triggered Emails in Email Studio



Triggered Send Definition in Email Studio

Querying for Content Assets

This API call was the first added to the cartridge because it is an easy HTTP “GET” call. It returns information about a content asset within SFMC.

Example calling

```
let queryResponse = marketingManager.QueryForContentAsset('32647');
```

SFMC Client Extension vs. Cartridge Addition

The examples provided in the above sections for each API call show the most likely use case, which would be extending the MarketingManager in the client’s core cartridge.
This could be done by inheriting the MarketingManager from this cartridge and using it as the base and adding new functions or it could be creating a file with a different name and/or location in the client’s core cartridge. Code in the client cartridge could then use it’s version of the MarketingManager class. The client’s version then calls the MarketingManager from this cartridge as needed.

Most of the client custom extension needs will come from encapsulating the retrieval of an external key or event definition key along with building out the data object into an easy to reuse function; “SendPasswordResetEmail(emailAddress)”, etc. The hardest part of communicating with SFMC is going to be making sure the data object passed is structured to match the expectation of SFMC; i.e. usually the data extension associated with a trigger or a journey.

Additions to this cartridge for Marketing Cloud should be driven by new API calls that need to be supported; for example, stopping a journey or using the new transactional messaging API (over the Triggered Send) if higher performance is needed.

Another possible addition to the client’s core cartridge, or possibly this cartridge if there is found to be a common enough use case, would be adding another “Manager” type file that would work with both clouds.
For example, after a customer creates their account in Commerce Cloud, the customer’s information should be sent to SFMC to begin their “Welcome Journey” and to SFDC to create a “Contact” for the customer.

Sales & Service Cloud

SFDC Configuration

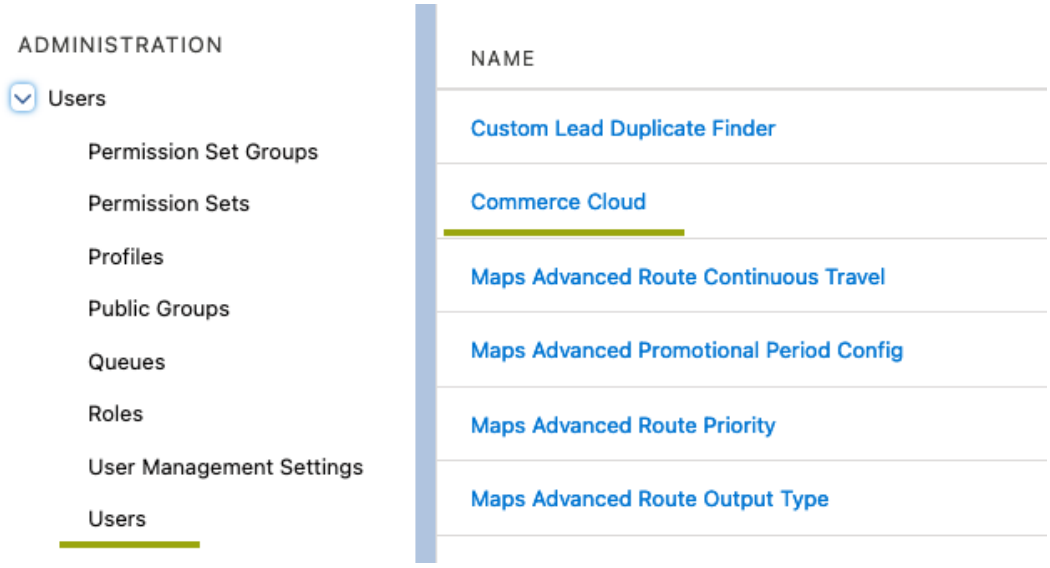
In order for Commerce Cloud to communicate with Sales / Service Cloud, the API integration has to be configured and enabled on SFMC.

Doing this requires administrative rights. Most clients, even smaller ones, will likely have a Salesforce administrator that may have already set up the API integration. However, if it is not set up and it needs to be done, the following steps maky be helpful in configuring the API integration.

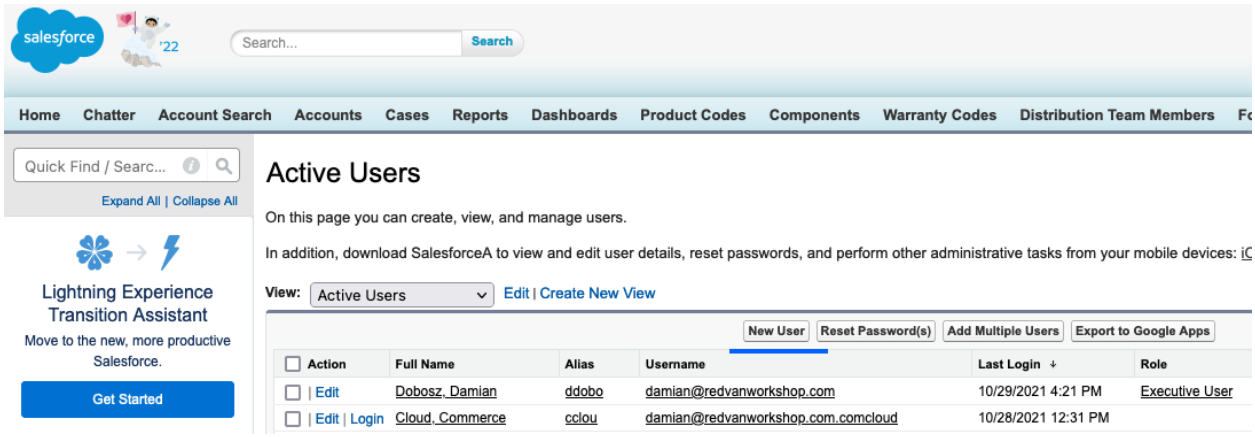
If it is already configured, then you can jump to the end of this section for the information on what you need from the client admin.

In SFDC

In order to be able to access existing configuration information or perform the configuration in SFDC, your account will need Administrative rights.
Navigate to the Users menu and if an account does not yet exist for the connected app, then create one.

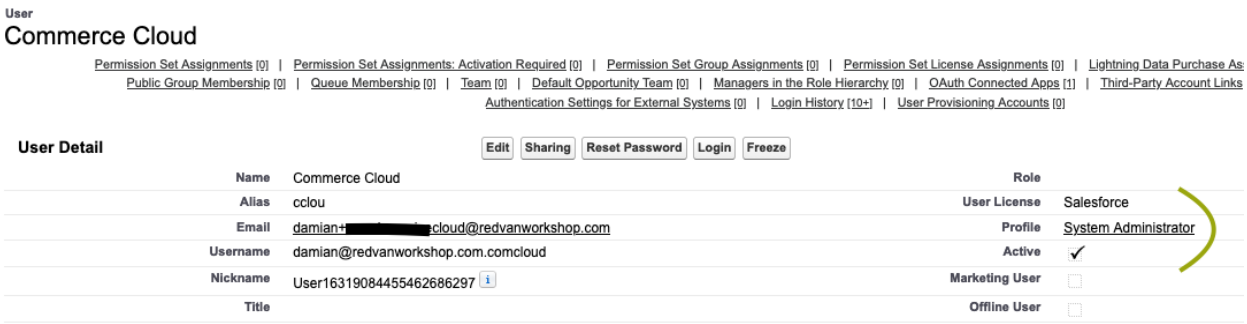


Administration - Users menu, recently used objects on the right



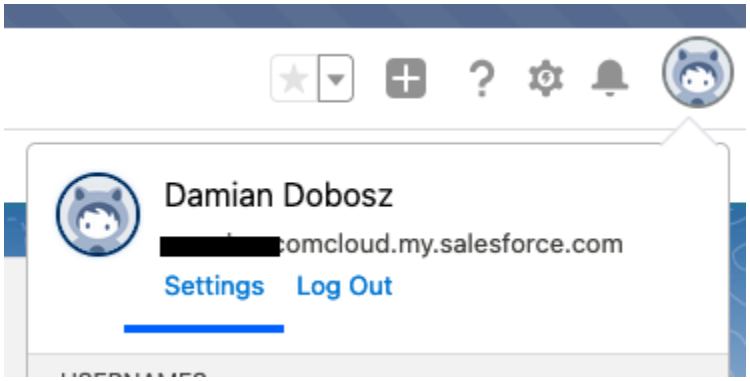
Create a New User in SFDC

When creating the user, it will need to have a Salesforce license, be an administrator and active.



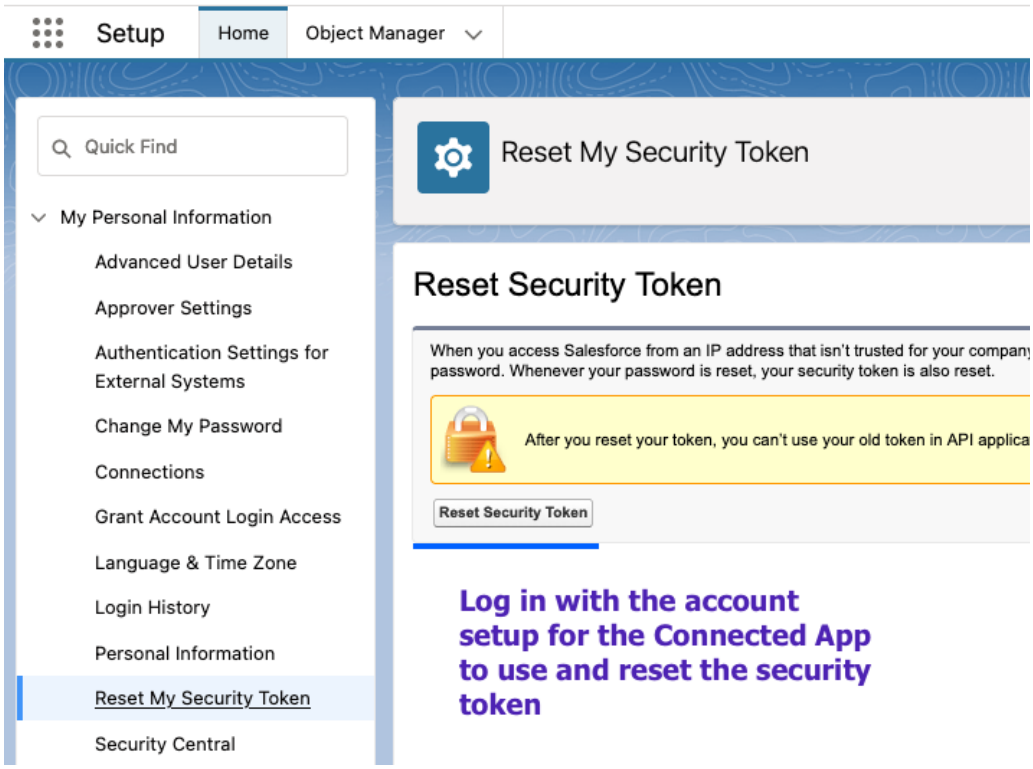
Service user in SFDC

Log in as the service account and then go to the account/profile settings page.



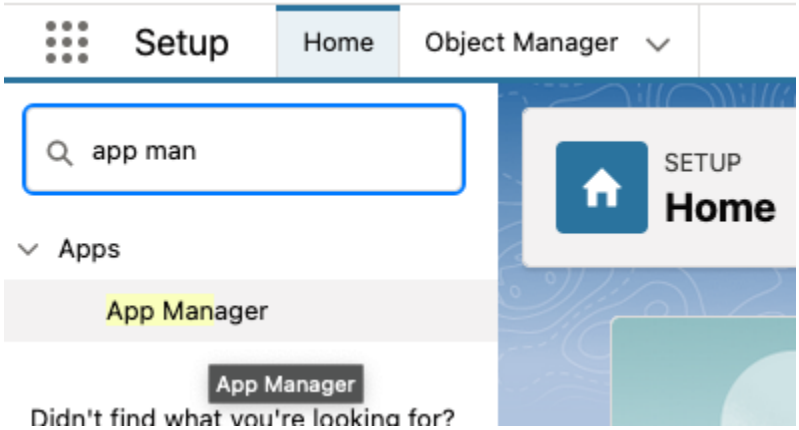
SFDC Settings menu

Select to reset the security token for the service account and record the value.



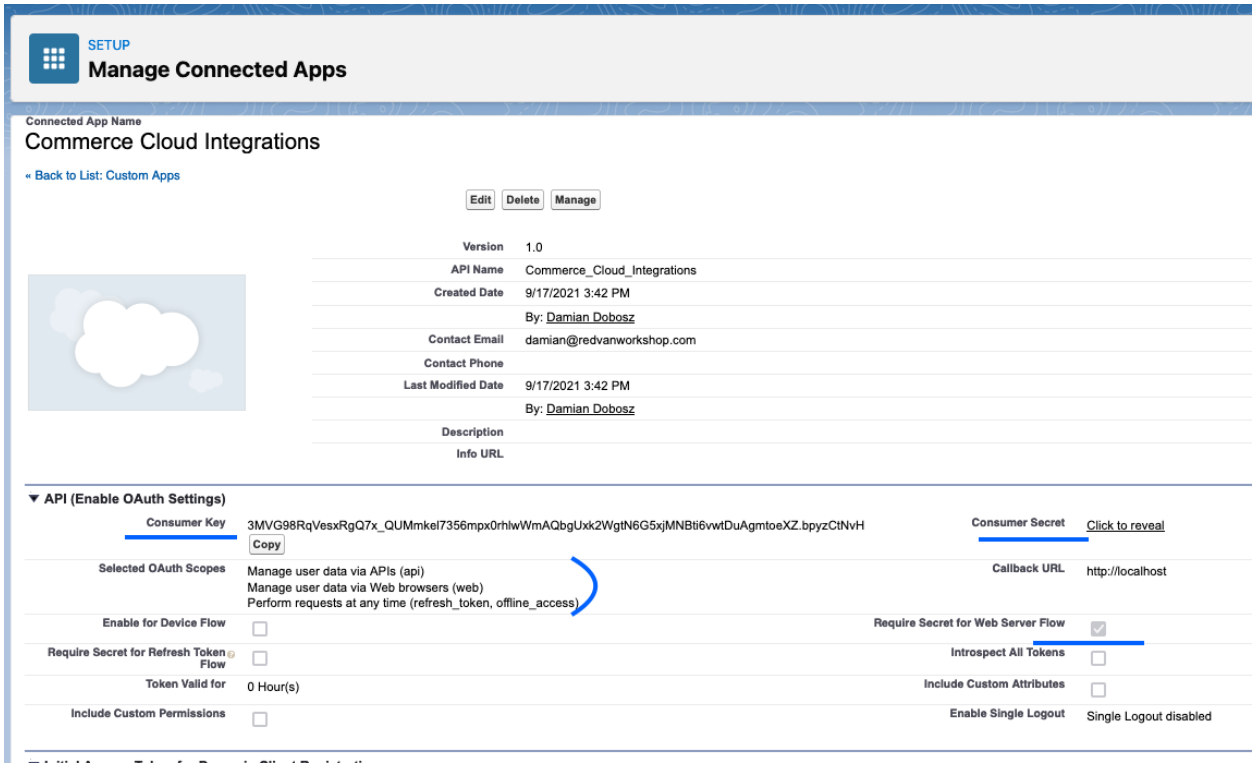
SFDC - Account Reset Security Token

Search for and open up the App Manager.



Searching for the App Manager in SFDC

If a Connected App does not already exist, create one. Make sure to select the listed OAuth scope (shown below) and to require the secret for the web flow.
Record the Consumer Key and Secret available here for use in SFCC.



Manage Connected Apps in SFDC

Also see this documentation in Red Van's Confluence for similar instructions.

SFDC API Integration Configuration Data Points

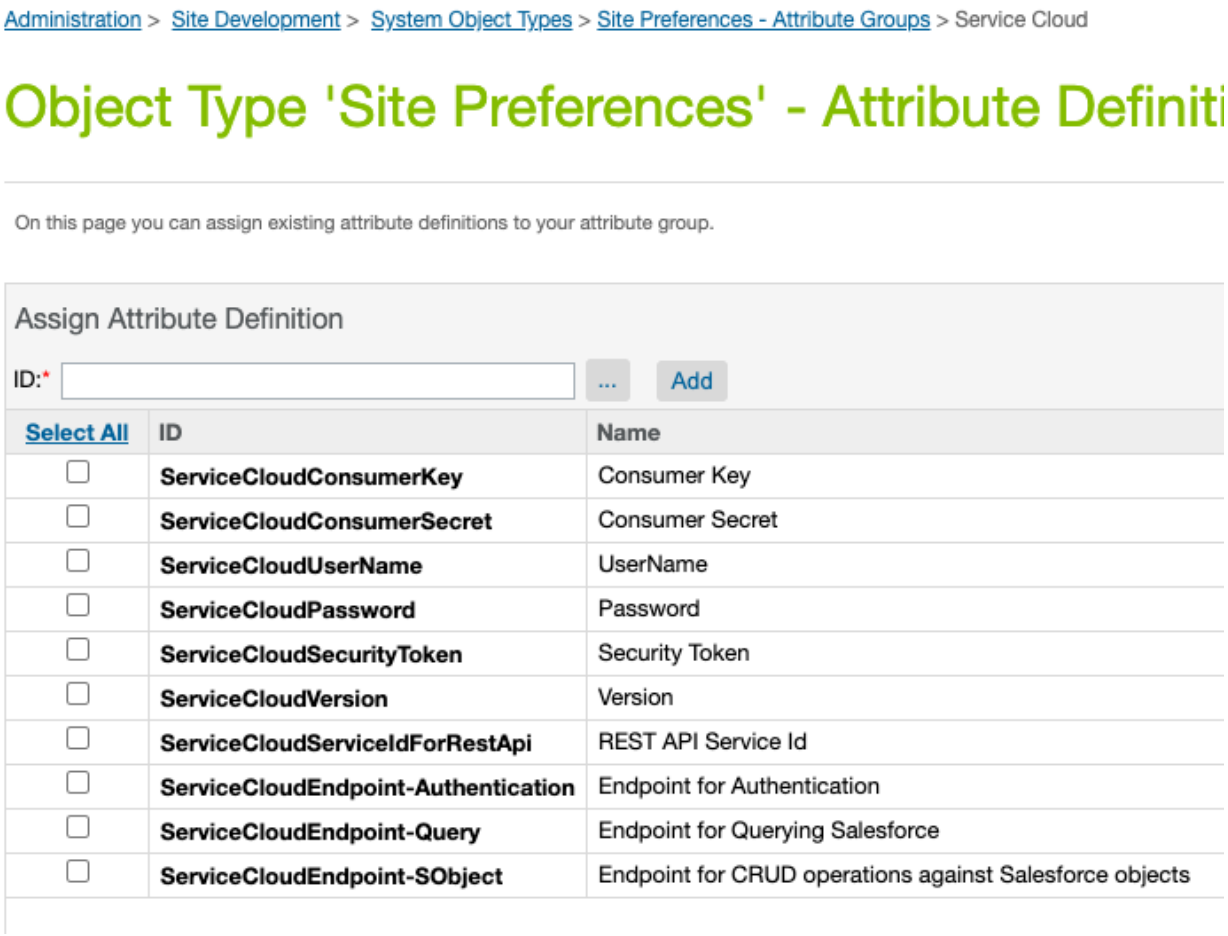
The data points needed by SFCC are:

- Consumer Key
- Consumer Secret
- User Name
- Password
- Security Token (for the user account)

SFDC In SFCC

SFDC Site Preferences

Of the 10 site preferences for Service Cloud, likely only 5 or 6 will need values for the specific SFDC instance that will be accessed.



Administration > Site Development > System Object Types > Site Preferences - Attribute Groups > Service Cloud

Object Type 'Site Preferences' - Attribute Definiti

On this page you can assign existing attribute definitions to your attribute group.

Assign Attribute Definition

ID:* ... Add

Select All	ID	Name
☐	ServiceCloudConsumerKey	Consumer Key
☐	ServiceCloudConsumerSecret	Consumer Secret
☐	ServiceCloudUserName	UserName
☐	ServiceCloudPassword	Password
☐	ServiceCloudSecurityToken	Security Token
☐	ServiceCloudVersion	Version
☐	ServiceCloudServiceIdForRestApi	REST API Service Id
☐	ServiceCloudEndpoint-Authentication	Endpoint for Authentication
☐	ServiceCloudEndpoint-Query	Endpoint for Querying Salesforce
☐	ServiceCloudEndpoint-SObject	Endpoint for CRUD operations against Salesforce objects

SFDC related Site Preferences

Those are:

- ServiceCloudConsumerKey
- ServiceCloudConsumerSecret
- ServiceCloudUserName
- ServiceCloudPassword
- ServiceCloudSecurityToken
- (Potentially) ServiceCloudVersion

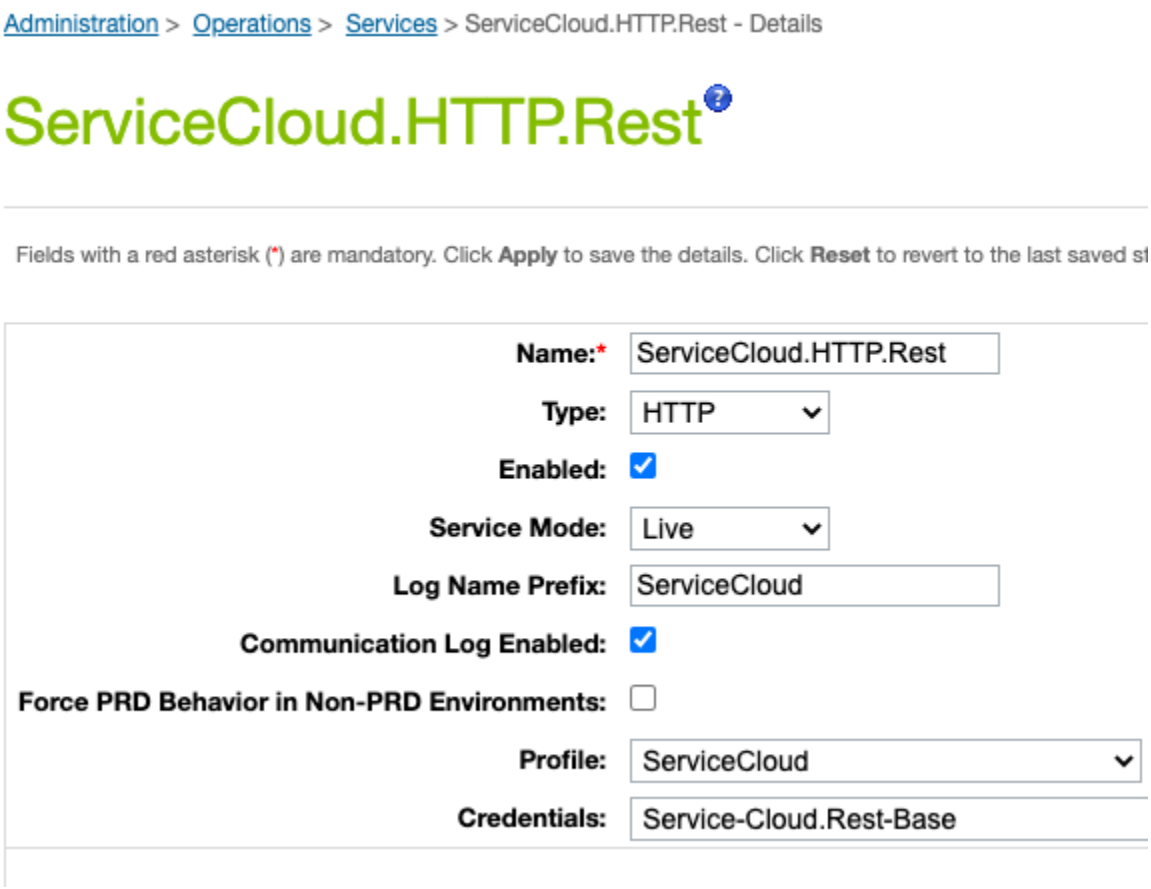
The value for the version will only need a value added over the default when the version in use changes at some point in the future due to a Salesforce update of the SFDC API and the version number that should be specified changes; e.g.. from “52.0” to “53.0”.

The remaining site preferences have default values that likely will not need to be changed unless there is an update to how the relative endpoint is structured in a future update to the SFDC API. For example, if the authentication endpoint were to change from “/oauth2/token” to “/oauth3/token”.

The site preferences for the ServiceIds should also not need to be changed as any configuration changes should be handled at the Service. See the next section for more details.

SFDC Services

Access to SFDC is handled by a single service definition in SFCC:



SFDC REST Service

If an SFCC instance should need to be switched to communicate with different SFDC instances, then a second credential object could be created with another URL.

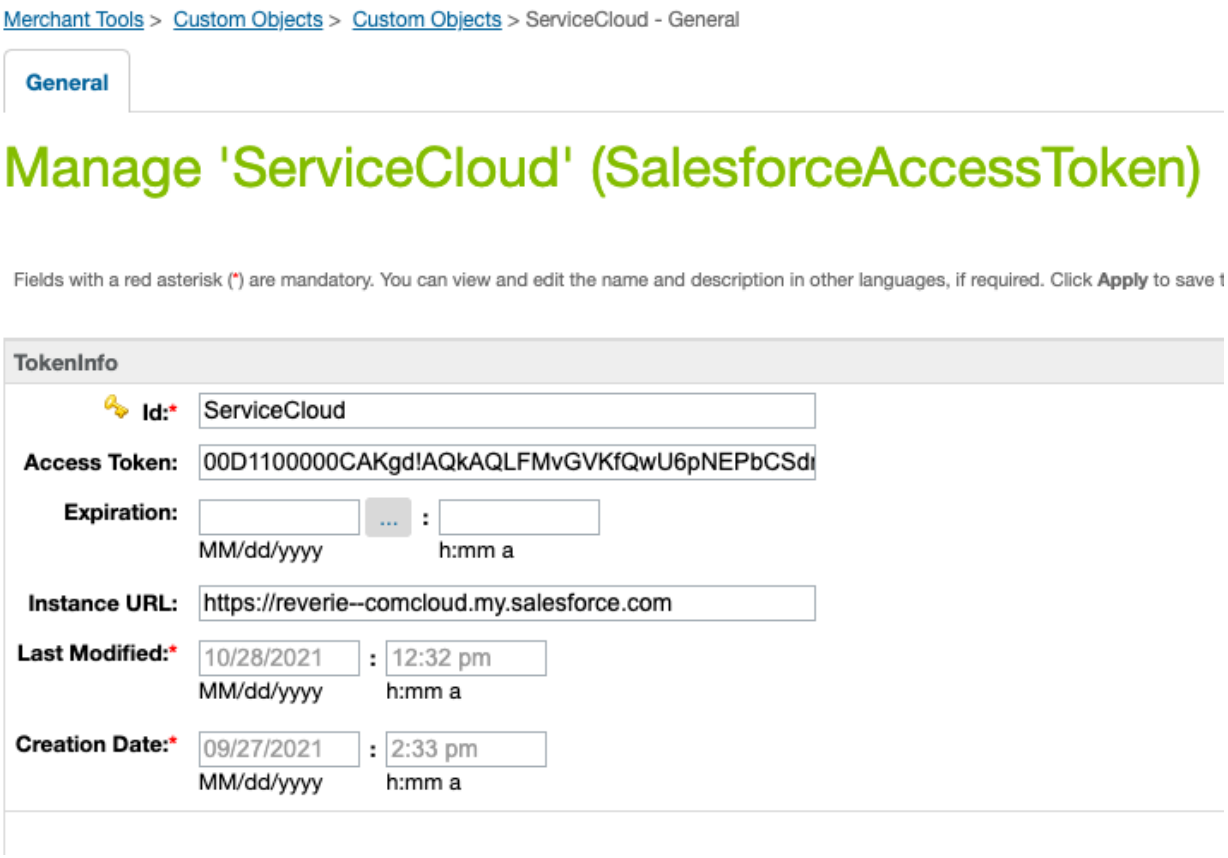
For example, included with the cartridge is a credential called "Service-Cloud.Rest.Base". A second credential called "Service-Cloud.Rest.Base.Production" or ".Test" (if on the production SFCC instance) could be created with the different URL.

It is worth noting here that the URL for the Service Cloud credential is likely to be "test.salesforce.com" or "www.salesforce.com" for sandboxes/development/staging or production. The authentication process will return the specific instance URL that should be used.

SFDC Instance of SalesforceAccessToken Custom Object

The metadata for this cartridge will also add the definition of a new custom object called “SalesforceAccessToken”. For more details on the definition of the custom object, see the details in the Marketing Cloud section.

An instance of this object on a site once the cartridge is in use will look like this:

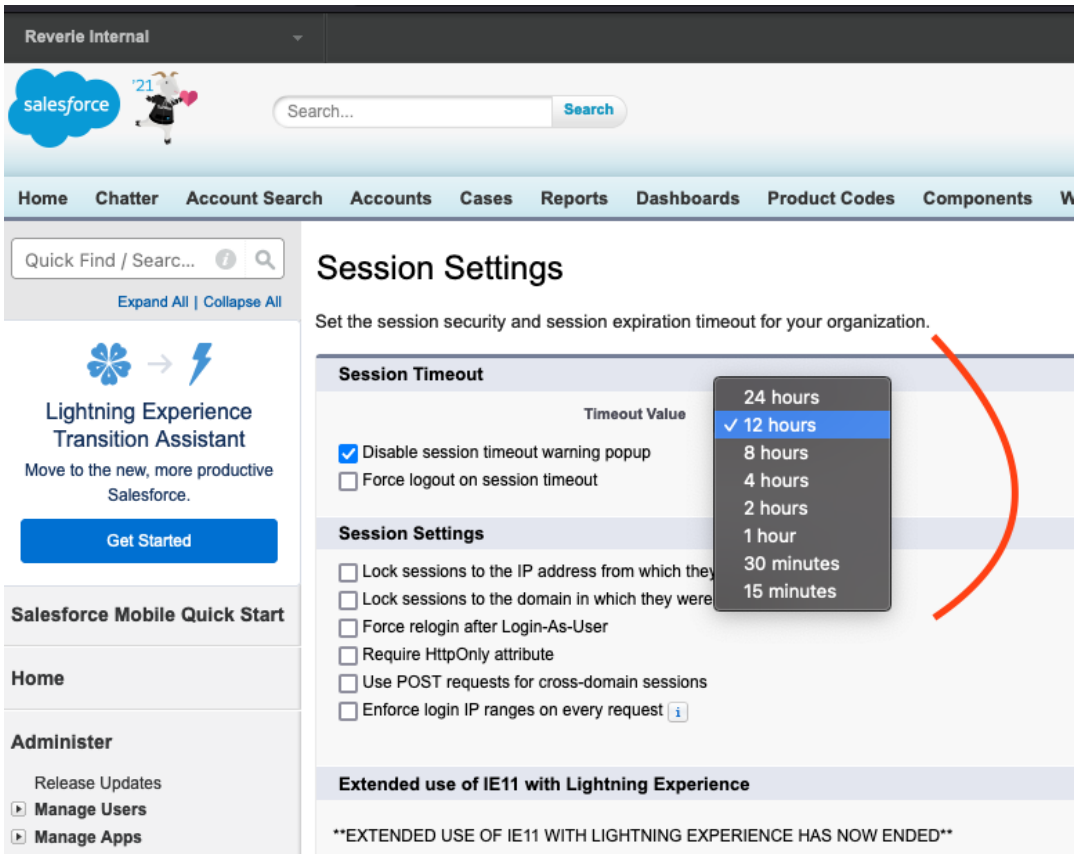


Instance of the SalesforceAccessToken for SFDC

When the metadata for this cartridge is imported, it will create an empty instance for the SalesforceAccessToken object for Service Cloud. It must exist prior to using the cartridge, as storefront requests in SFCC cannot create instances of custom objects, and an instance of it is needed to store the access token SFDC returns during the authentication process.

It is also worth noting, that SFDC does not expire tokens in the same way that SFMC does, nor does it return an expiration date time with the authentication request.

Within SFDC, the session timeout settings can be customized as the following screenshot shows:



SFDC Session settings controlling token timeout

Not only can the token (session) expiration be set to be considerably longer than the 20 minutes that the SFMC token life is, it also is a rolling timeout, meaning, that as long as the token is being used, the timeout will be moved forward. It is only if the session (token) were to be idle for 15, 30 minutes, 1 hour... up to 24 hours (whatever it is set to) that the token would expire.
The authentication and API process is set up to automatically handle token expiration and renewal. See the Supported API section (next) for more details.

SFDC Supported API

The following sections detail which API calls are currently supported and may include additional information on common use cases, examples of calling them and possible extension points.

For the examples in all of the following sections, this is the require statement that would also be needed:

```
var serviceManager = require('*/cartridge/scripts/service/ServiceManager');
```

SFDC Authentication

There should be no need to directly interact with SFDC authentication. It may be worth knowing that the access token returned from SFDC will only expire if the service usage becomes idle for longer than the session timeout configuration in SFDC.
The cartridge will automatically handle retrieving a new token when it is needed.

Query

The Query API call is an HTTP GET that modifies the URL of the service call to include the query. Because a query can be made of any standard or custom object, there are several functions provided on the ServiceManager class for querying; there is a generic “select by attribute” where the caller passes the select, the object and the attribute and value.
For example, that could be 'FIELDS(ALL)', 'Case', 'Id' to select all the fields from the Case object (table) by the Id.

Additionally, other simplified and specific functions are provided for querying against different standard, common Salesforce objects; specifically “Case” and “Lead”.
It is foreseeable that more of these types of functions will be added for common use case queries in SFDC.

Common Use Case: Client wants to query if a Lead already exists with a given email address before adding to the Lead object (table).

Example Calling

```
let leadInfo = {
      "FirstName": "Hank",
      "LastName": "McHankerson",
      "Email": "hank@mailinator.com",
      "LeadSource": "Web"
};
let leadQueryResponse = serviceManager.QueryForLeadByEmail(leadInfo.Email);
```

Possible Client Extension
A foreseeable client side extension would be to add similar specific query functions for custom Salesforce objects to the client’s core cartridge;
e.g. “QueryXyzCustomObjectByEmail(email)”
Which would then call the generic query function on the ServiceManager class.

Additionally, custom functions that combine SFDC API calls may be useful to a client. For example, in a “ServiceManager” file in the client’s core cartridge, first query if an object exists, then if not, create that object.

Create Object

The Create Object API call is an HTTP POST that modifies the URL of the service call to include the object being created.. Because the create object call can be made for any standard or custom object, there are several functions provided on the ServiceManager class for creating objects; there is a generic “create any object” where the caller passes in which type of object to create and the data for creating it.

For example, that could be “Lead” and a JSON object with the key value pairs of data to insert into the Lead object (table).

Additionally, there is a combination function provided that accepts Case and Contact information and will get or create a contact to match the given contactInfo, and then will create a case linked to that contact.
This could also be known as a “Web to Case” use case.
It is foreseeable that more of these types of combination functions will be added for common use case object creation in SFDC as well as “Update” and “Delete” object functions.

Common Use Case: Client wants to create a Case using the given information entered by the customer on the web site; i.e. “Web to Case”.

Example Calling

```
let contactInfo = {
      "FirstName": "Anya",
      "LastName": "Cabanya",
      "Email": "cabanya@mailinator.com",
      "Phone": "303.555.4444",
      "LeadSource": "Web"
};
let caseInfo = {
      "Subject": "Warranty Case created by Customer",
      "Description": "Product XYZ broke after second use.",
      "Serial_Number_w2c__c": "20210203-07998001",
      "Product_Code_w2c__c": "A320-S",
      "Date_of_Sale_w2c__c": "2021-10-25"
};
let createCaseResponse = serviceManager.CreateCaseWithContact(caseInfo,
contactInfo);
```

Possible Client Extension
A foreseeable client side extension to the client's core cartridge would be to add similar combination functions that work with custom Salesforce objects or standard Salesforce objects in a client specific use case;
e.g. "CreateXyzCustomObjectAndCase(xyzInfo, caseInfo)"

A function like this would then use the generic create object function on the ServiceManager class for each object being created..

SFDC Client Extension vs. Addition

The examples provided in the above sections for each API call show the most likely use case, which would be extending the ServiceManager in the client's core cartridge.
This could be done by inheriting the ServiceManager from this cartridge and using it as the base and adding new functions or it could be creating a file with a different name and/or location in the client's core cartridge. Code in the client cartridge could then use it's version of the ServiceManager class. The client's version then calls the ServiceManager from this cartridge as needed.

Client custom extension needs will likely come from combining actions, such as query to find an existing Asset, Account, Contact or Lead and retrieve the relevant SFDC Id's for the those object(s), then use those Id(s) to create a new Case or other object.

The hardest part of communicating with SFDC is going to be making sure the data object passed is structured to match the expectation of SFDC. For standard objects, that data is available in Salesforce documentation; e.g. Contacts.
For custom objects, that will likely require working with the client's SFDC admin or RVW SFDC Architect assigned to the client to confirm the data structure.

Additions to this cartridge for Service Cloud should be driven by new API calls that need to be supported; for example, operations such as Update or Delete against existing Salesforce objects.

Additionally, common combination functions that are still generic, i.e. usable in most/all SFDC environments; e.g. retrieve or create an Account, the create a Contact linked to that Account.

Another possible addition to the client's core cartridge, or possibly this cartridge if there is found to be a common enough use case, would be adding another "Manager" type file that would work with both clouds.

For example, after a customer registers their product on the Commerce Cloud site, that information should be added to SFDC as an Asset and Contact, and that information should also be sent to SFMC so the customer gets a "Product Registration Received" email.